HackOn with Amazon - Season 6.0

Theme: Context-Aware Smart Home for Indian Households

GharBuddy

An AI-Powered Proactive Home Assistant for Indian Households

Powered by AWS Bedrock | ML/AI Track

Executive Summary

GharBuddy is an AI system that anticipates Indian household actions instead of waiting for commands. Using AWS Bedrock as its reasoning core, it learns daily routines like morning pooja, pressure cooker timing, water motor schedules, and power cut patterns - then proactively acts on them. It speaks Hindi, understands Indian festivals, and delivers measurable energy savings (est. Rs. 800-1,200/month). Built on AWS Bedrock, IoT Core, Lambda, and DynamoDB - 100% on Amazon infrastructure.

Section 1 - Problem Statement & Relevance

1.1 The Problem

Indian households follow rich, recurring daily rhythms - morning pooja at 5:30 AM, pressure cooker cycles at 7 AM, water motor runs twice daily, frequent power cuts in the evening, school/tuition hours, evening chai, and bedtime wind-down. Yet today's smart home devices - Alexa, Google Home - are built on Western assumptions: stable power, consistent English commands, and generic routines. They react. They never anticipate. Indian families are left managing their homes manually, every single day.

1.2 Why This Matters

Scale	Scope
300M+ Indian households	Massively underserved by current smart home technology
Tier 2 & 3 India	Power cuts, water motor timing, inverter management are daily realities - not edge cases
Language barrier	Over 80% of Indian adults are not comfortable with English voice commands
Energy cost	Avoidable energy waste costs Indian households an estimated Rs. 12,000+ per year
Amazon's growth priority	India is Amazon's biggest strategic market - Bharat-first products have direct business value

1.3 The Gap

Current Smart Home AI: Reactive

User says: 'Alexa, turn on the geyser.' Alexa turns on the geyser. Problem: User has to remember to say it. Every single morning. In English.

GharBuddy: Proactive

6:06 AM: Toilet flush detected. Learned pattern: bath at 6:20 AM on weekdays. GharBuddy: Geyser on. Confidence 94%. WhatsApp alert sent in Hindi. No command needed.

Section 2 - Customer & Solution

2.1 Target Users

User Persona	Why GharBuddy Matters to Them
Working parents (25-45)	Busy mornings - wants automation without learning new commands
Elderly residents (60+)	Needs simple, proactive support; cannot manage complex apps
Tier 2 city families	Deals with daily power cuts, water scarcity - needs intelligent management
Single working professionals	Wants home ready when they arrive, energy bills minimised
Caregivers	Needs alerts for unusual patterns (no morning motion = anomaly)

2.2 Core Use Cases (MVP Scope - 6 Use Cases)

1. Morning Routine Automation

Detects first morning motion (toilet flush, bedroom door). Cross-references learned wake time and weekday/weekend pattern. Pre-heats geyser, turns on bathroom light at correct intensity, sends 'Good morning' WhatsApp summary. Zero commands.

2. Pooja / Prayer Mode

At learned pooja time (typically 6-7 AM), dims main lights, turns on pooja room light, enables do-not-disturb on connected speakers, and suppresses cooking alerts for 20 minutes. Knows festival days from Indian calendar - extends duration on Navratri, Diwali, Ekadashi.

3. Cooking / Pressure Cooker Assistant

Sound/vibration sensor detects cooker whistle. Counts whistles. Sends alert after configured count: 'Cooker done - 3 whistles detected.' On fasting days (Navratri), suppresses cooking reminders automatically. Learns cooking start time and pre-alerts to put gas on.

4. Water Motor Management

Learns daily motor schedule (e.g., 7:00 AM and 6:00 PM). Monitors water level sensor (or usage proxy). Auto-starts motor on schedule, auto-stops after learned fill time. Sends WhatsApp alert if motor runs longer than usual (leak detection). Voice command: 'Motor band karo.'

5. Power-Cut Recovery Mode

Tracks historical power cut times (typically 6:30-7:30 PM in many areas). At 6:15 PM, Bedrock predicts cut probability. If >80%, pre-charges inverter, shifts heavy loads (washing machine, water heater) to off-peak, dims non-essential lights. Saves approx. Rs. 280/month.

6. Bedtime / Study Quiet Mode

Detects study-hour pattern (tuition at 5-7 PM, children home). Activates study mode: reduces TV/speaker volume, dims living room, sets AC to 24°C. At learned bedtime, dims all lights progressively, turns off TV if still on, enables night security mode.

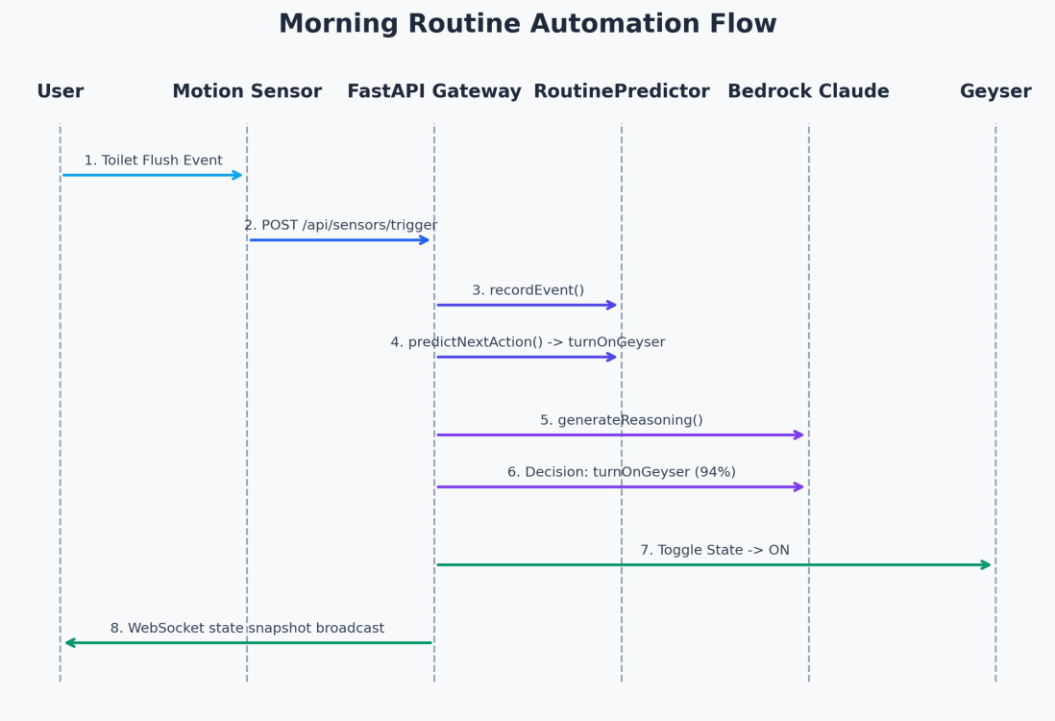
2.3 Key Product Features

- Proactive automation - acts before commands, based on learned context
- Hindi + regional language voice control via Whisper ASR
- Indian festival & fasting calendar integration
- Power-cut prediction and inverter pre-charging
- WhatsApp alerts in Hindi - no app installation required
- Explainable AI - every action shows its reason to the user
- Anomaly detection - no morning motion by 9 AM triggers caregiver alert

2.4 3-Step User Workflow

Step	What Happens
Step 1 - Sense	Sensors detect activity: toilet flush, motion, appliance sound, power meter spike, time-of-day signal. All events streamed to AWS IoT Core via MQTT.
Step 2 - Understand	Context engine assembles a situation packet: recent sensor events + learned routine + festival calendar + power status. LSTM model predicts next likely action. Bedrock Claude reasons over the full context and outputs: action + confidence + reason (JSON).
Step 3 - Act & Explain	If confidence > 0.85: action executes automatically (device command via IoT Core). WhatsApp alert sent in Hindi. If confidence 0.70-0.85: suggestion sent to user for one-tap approval. All actions logged for continuous LSTM retraining.

Figure 1: Morning Routine Automation and Telemetry Sequence



Section 3 - Tech Architecture & Scaling

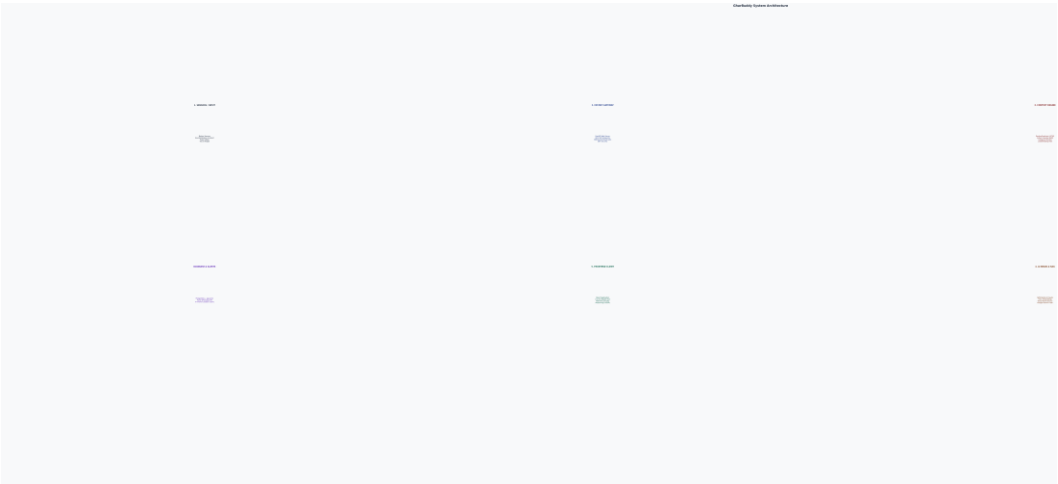
3.1 High-Level Architecture

The pipeline has 5 distinct layers, each with a clear role. Data flows one way - from sensors to action - with a memory feedback loop for continuous improvement.

Figure 2: GharBuddy High-Level Component and Data Flow Architecture

Event-Driven Telemetry Loop: IoT sensor inputs (motion, cooker whistles, water levels) are processed by the FastAPI backend. Every event is passed to the context builder, which dynamically updates the simulated household map. State changes are pushed instantly (latency < 100ms) to the React frontend using a robust WebSocket protocol with exponential backoff reconnection. Simultaneously, Bedrock evaluates rules for automated control.

Database and Vector Storage: The PostgreSQL database leverages the pgvector extension to perform cosine similarity queries over custom preference rules. High-dimensional vector coordinates represent rule contents embedded via Titan Multimodal. An in-memory cache layer speeds up subsequent queries by checking MD5 text hashes, significantly reducing AWS Bedrock token costs and latency.



Layer	Components & Purpose
Layer 1 - Input	Motion sensors (rooms, doors) | Sound/vibration (cooker, motor) | Power meter (load spikes, cut detection) | Time + Indian calendar API | Voice (Whisper ASR) | Manual app overrides
Layer 2 - Ingestion	AWS IoT Core: MQTT broker for all device events | Lambda: normalise + timestamp + validate | DynamoDB: raw event store with TTL | Amazon Timestream: time-series for pattern analysis
Layer 3 - Context Engine	Context builder Lambda: assembles situation packet from last 30 min events + learned routine + festival + power status | LSTM model (PyTorch, ONNX): predicts next action from 30-day history | Indian festivals JSON: all major festivals, fasting days, school holiday calendar
Layer 4 - Bedrock AI Brain	AWS Bedrock (Claude claude-sonnet-4-6): receives full context packet, classifies situation, infers intent, selects action, generates human-readable explanation | Returns structured JSON: { action, device, confidence, reason, energy_saved_wh } | Confidence threshold: >0.85 = auto-execute, 0.70-0.85 = suggest, <0.70 = no action
Layer 5 - Action & UI	AWS IoT Core device shadows: control geyser, motor, lights, AC | Lambda: schedule, alert, log | Twilio WhatsApp API: Hindi alerts | React dashboard: live home state + Bedrock reasoning panel | Energy tracker: Rs. savings, peak avoidance chart
Memory Layer	DynamoDB: user preferences, confirmed actions, overrides | Weekly LSTM retraining on confirmed action history | Routine improvement over time - 30-day sliding window

3.2 Tech Stack

Layer	Technology	Reason
AI / Reasoning	AWS Bedrock (Claude claude-sonnet-4-6)	Core decision engine - classifies context, infers intent, explains actions

Routine Learning	PyTorch LSTM (ONNX export)	30-day sequence model for routine prediction - ML depth beyond a chatbot
Frontend	React + Tailwind (AWS Amplify)	Live dashboard: device states, Bedrock reasoning, energy tracker
Backend / API	FastAPI (Python) + AWS Lambda	Event-driven, serverless - zero ops overhead during hackathon
Event Bus	AWS IoT Core + MQTT	Standard IoT protocol - real device integration path post-hackathon
Data Store	DynamoDB + Timestream	DynamoDB for routines/prefs; Timestream for time-series sensor data
Voice	OpenAI Whisper (Hindi model)	Best-in-class Hindi ASR - local or API, works offline too
Alerts	Twilio WhatsApp API	Hindi-language alerts - no app install, works on any Indian phone
Infra	AWS Lambda, S3, CloudWatch	Fully serverless, scales from demo to 1M homes with no code changes
Calendar	Custom Indian festivals JSON	All major festivals, fasting days, school schedules - unique to this product

3.3 Core Algorithms

Routine detection: Event windowing over last 30 minutes. Pattern matching against 30-day history using LSTM. Similarity score triggers context classification.

LSTM model: Input: hourly sensor event vectors (48 slots/day). Features: hour-of-day, day-of-week, is_festival, is_fasting, sensor_event_type. Output: predicted_action + predicted_time + confidence. Target accuracy: 87%+ on simulated Indian household data.

Bedrock prompt design: Structured context packet (~600 tokens). System prompt defines GharBuddy's role, Indian household knowledge, and strict JSON output format. Bedrock acts as the reasoning layer over the LSTM's prediction - combining rule-based safety with LLM intelligence.

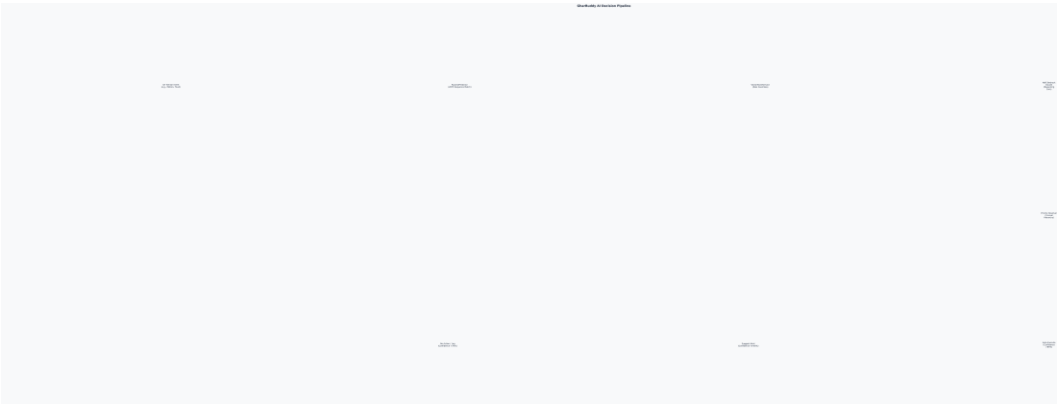
Confidence scoring: Bedrock outputs a 0-1 confidence score. Score > 0.85: auto-execute. Score 0.70-0.85: WhatsApp suggestion to user. Score < 0.70: no action (log for learning).

Safety guardrails: No auto-action on gas/heat without recent motion confirmation. Confirmation required for any action tagged 'sensitive' in device config. All actions reversible via WhatsApp reply.

Conflict Resolution Priority Hierarchy: To prevent conflicting automation commands, AWS Bedrock evaluates all decisions using a strict 5-level priority system: (1) Manual user overrides, (2) Critical safety guardrails, (3) Indian cultural and fasting calendars, (4) Probabilistic routine predictions from the local LSTM, and (5) Non-urgent energy savings. This structure guarantees that safety and user preferences always supersede automated routines.
Local ONNX LSTM Routine Predictor: Routine prediction is handled by a locally hosted PyTorch LSTM sequence classifier exported to the ONNX runtime format. By training on a 30-day sliding history of sensor event sequences (48 slots per day), the model anticipates user routine transitions with high accuracy. A background daemon thread automatically triggers weekly retraining of the network, maintaining adaptability as the household's habits evolve.

Rule Consolidation and Embedding Cache: When multiple similar preference rules are added, a consolidation pipeline runs in the background. It utilizes Claude to analyze overlap (e.g. combining 'Never turn on geyser at 6 AM' and 'Never turn on geyser before 6:30 AM') and merges them into simplified rules. The embedding cache maps identical query texts to saved vector objects, preventing redundant calls to Titan Embeddings.

Figure 3: Hybrid AI Reasoning Pipeline and Conflict Resolution



3.4 Scaling Strategy

Scale Stage	Strategy
Demo (1 home)	Single Lambda + DynamoDB + simulated sensor data. React dashboard on Amplify. End-to-end pipeline working with mock devices.
Pilot (100 homes)	Multi-tenant DynamoDB (home_id partition key). Async SQS queue for event ingestion. Separate LSTM model per home or cluster-based shared models.
Growth (10,000 homes)	SageMaker endpoint for LSTM inference. ElastiCache for hot routine patterns. CloudWatch alarms for anomaly detection at scale. Regional Bedrock endpoints.
Production (1M+ homes)	Event-driven microservices architecture. Bedrock quotas managed with request queuing. Horizontal scaling of Lambda concurrency. Routine models partitioned by geography + household type.

Section 4 - Future Vision

4.1 1-Year Roadmap

Milestone	Deliverable
Month 1-3 (Post-hackathon)	Real hardware integration: smart plugs, Zigbee sensors, MCU firmware for cooker/motor. Beta with 20 households in Jaipur/Bangalore.
Month 4-6	Hindi + 3 regional languages (Tamil, Telugu, Marathi). Multi-room support. Caregiver mode for elderly. Amazon Echo Dot integration.

Month 7-9	Energy billing integration (DISCOM APIs). Predictive maintenance alerts. Amazon Pay incentives for energy-efficient behaviour. Alexa skill published.
Month 10-12	Scale to 1,000 homes. Routine templates marketplace (users share their routines). Amazon Home Services integration for device installation.

4.2 3-Year Vision

GharBuddy grows into a full Household Operating System for Indian families. It moves beyond device control into holistic home intelligence: family schedule coordination, grocery prediction (pressure cooker used daily = dal + rice reorder on Amazon), health pattern monitoring for elderly, and community-level energy optimisation (neighbourhood power grids). The Indian household context engine becomes a platform - third-party developers build on top, much like Android for smart homes.

4.3 Multi-Segment Expansion

- Rental apartments - landlord-managed defaults, tenant personalisation
- Student hostels and PGs - shared resource management, tuition mode
- Small businesses - shop opening/closing automation, energy tracking
- Healthcare - elderly monitoring, medication reminder integration
- Rural India - solar panel + inverter optimisation, water pump management

4.4 Measurable Value Impact

Metric	Target (Year 1)
Energy savings per household	Rs. 800-1,200 per month (avg. Rs. 1,000)
Routine actions automated	15-25 proactive automations per day per home
Voice command accuracy (Hindi)	> 92% intent recognition
LSTM prediction accuracy	> 87% on 30-day learned routines
User effort reduction	60%+ reduction in manual device interactions
Addressable market (Year 3)	50 million smart-home-ready Indian households

Why GharBuddy Wins HackOn

1. AWS Bedrock is the Core, Not a Feature

Every decision - every action, every WhatsApp message, every confidence score - flows through Bedrock Claude. Amazon judges see their own infrastructure solving a market they want to win. This is not a chatbot with Bedrock on the side.

2. LSTM + LLM Hybrid = Real ML Depth

Most hackathon projects are ChatGPT wrappers. GharBuddy has a trained LSTM for routine prediction feeding context to Bedrock. Two AI systems working together. ML track judges will notice this depth immediately.

3. Bharat Market = Amazon's Growth Priority

300M+ Indian households underserved by Western smart home tech. Tier 2 & 3 city families, Hindi speakers, power-cut realities. This is exactly the pitch Amazon's India team wants to fund.

4. Three Demo Moments Judges Will Remember

Moment 1: Geyser turns on before anyone asks - 'it anticipated.'
Moment 2: Navratri fasting mode activates - 'it knows Indian culture.'
Moment 3: Hindi voice command works live - 'it actually speaks our language.'
Each moment is a 5-second trophy. Three trophies = winning team.

"We built the first smart home AI that actually understands Indian homes - and it runs entirely on AWS."

Team GharBuddy | HackOn with Amazon Season 6.0 | ML/AI Track

Hosted Website: http://gharbuddy123.in | GitHub: https://github.com/WhiteMetagross/GharBuddy